Uso de Visual Paradigm

Benjamín Morales

Creación de la clase Estadisticas

La nueva clase se creó directamente en el diagrama, dentro del paquete Modelo. Su responsabilidad es calcular métricas a partir de una lista de objetos Resultado.

Los atributos son todos privados, estos siendo la lista de los resultados y las estadísticas que se registraron. Las operaciones públicas son el constructor y un getter por cada métrica: getTotalJugadas(), getVictorias(), getPorcentajeVictorias(), getRachaMaxima() y getTipoMasJugado().

Para decidir las relaciones se analizó con qué clases interactúa Estadisticas y si el vínculo es permanente o puntual:

- Estadisticas se asocia con la clase Resultado debido a que mantiene una lista de resultados respecto a un usuario en específico.
- El tipo más jugado se guarda como un objeto de la clase TipoApuesta y asi Estadisticas se asocia con la clase TipoApuesta.

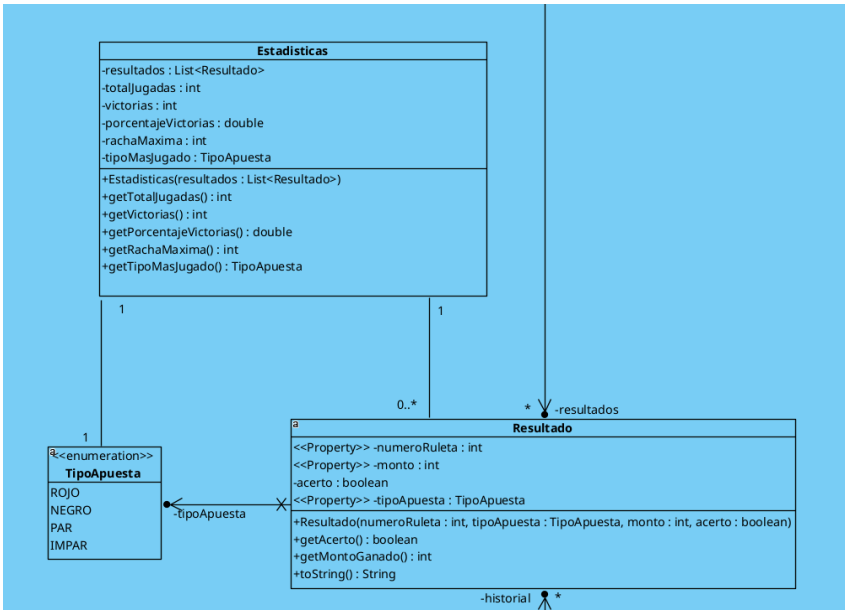


Figura 1. Creación de la clase Estadisticas en Visual Paradigm

Con el diagrama terminado se generó el código inicial desde Visual Paradigm utilizando el proceso de Round-Trip Generate Code, usando la opción de merge para conservar el código original y agregar la nueva clase.

Code Blame 41 lines (32 loc) · 759 Bytes

```java
package Modelo;

public class Estadisticas {

    private List<Resultado> resultados;
    private int totalJugadas;
    private int victorias;
    private double porcentajeVictorias;
    private int rachaMaxima;
    private TipoApuesta tipoMasJugado;

    /**
     *
     * @param resultados
     */
    public Estadisticas(List<Resultado> resultados) {
        // TODO - implement Estadisticas.Estadisticas
        throw new UnsupportedOperationException();
    }
```

Figura 2. Fragmento de la clase Estadisticas generada por Visual Paradigm

A continuación se implementó mediante el IDE IntelliJ IDEA un nuevo controlador nombrado EstadisticasController, el cual va a ser el comunicador entre la vista del menú y los datos obtenidos del objeto de Estadisticas.

```java
package Controlador;

import Modelo.Estadisticas;

public class EstadisticasController {  3 usages  BevelBen
    private final Estadisticas estadisticas;  6 usages

    public EstadisticasController(SessionController sessionController) {  1 usage  BevelBen
        this.estadisticas = new Estadisticas(sessionController.obtenerHistorial());
    }

    public String obtenerEstadisticas() {  1 usage  BevelBen
        return "Total Jugadas: " + estadisticas.getTotalJugadas()
                + "\nVictorias: " + estadisticas.getVictorias()
                + "\nPorcentaje de Victorias: " + estadisticas.getPorcentajeVictorias()
                + "\nRacha Maxima de Aciertos: " + estadisticas.getRachaMaxima()
                + "\nTipo de Apuesta Más Jugada: " + estadisticas.getTipoMasJugado().name();
    }
}
```

Figura 3. Estructura de la clase EstadisticasController

Terminada la implementación, se volvió a Visual Paradigm y se ejecutó operación Round-Trip Reverse Code sobre el diagrama, para que los cambios hechos en IntelliJ IDEA quedarán reflejados en el modelo. El diagrama actualizado incorpora

los métodos privados de cálculo (calcularTotalJugadas(), calcularVictorias(), calcularPorcentajeVictorias(), calcularRachaMaxima() y calcularTipoMasJugado()) y la asociación de la clase con TipoApuesta a través del atributo tipoMasJugado.

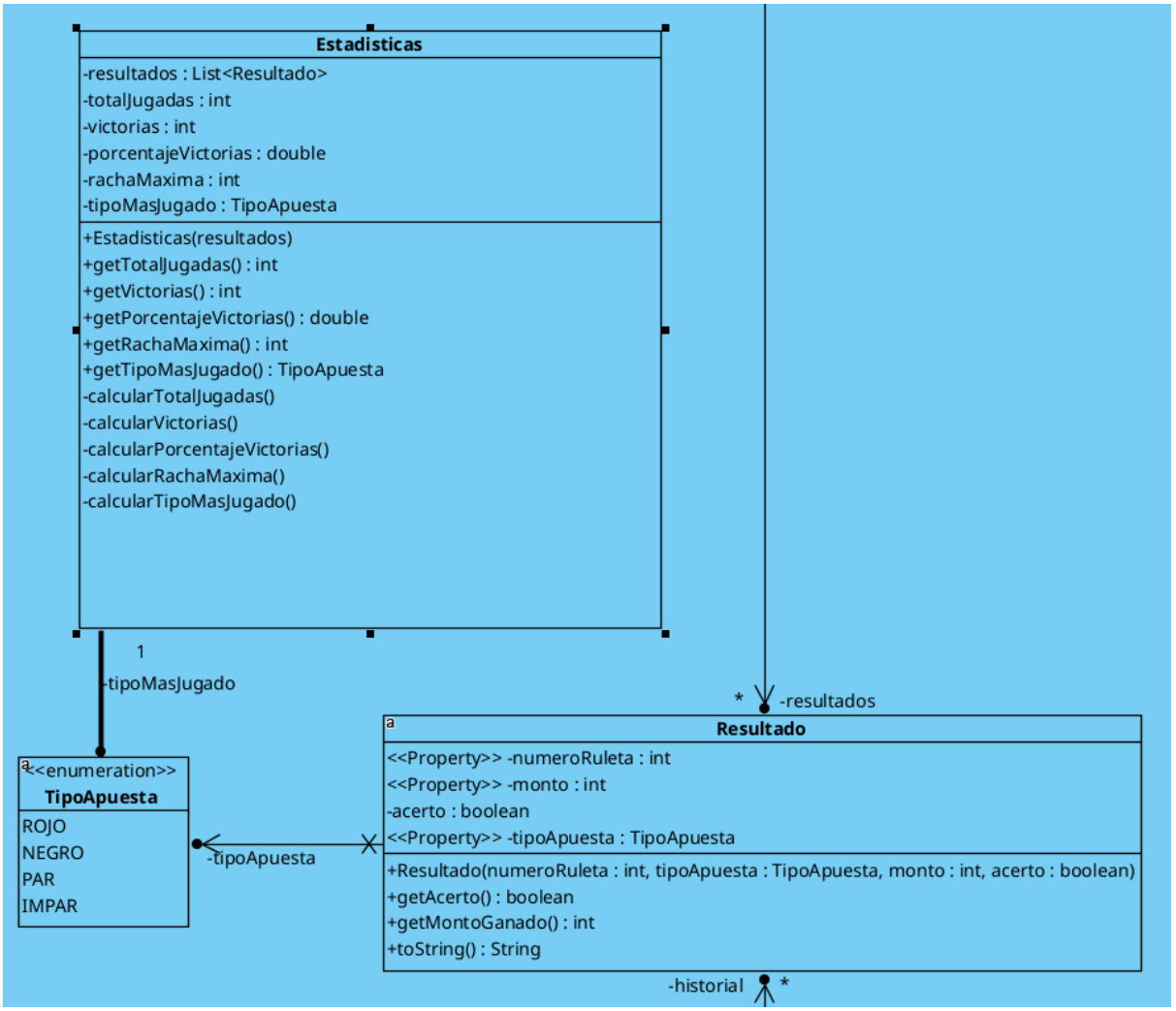


Figura 4. Diagrama UML actualizado en Visual Paradigm.